Stela Platform Architecture: Governance Hardening and Pointer-First Agent Constitution (DP-OPS-0042 Analysis)

1. Executive Summary

The evolution of the Stela repository—a stateless, repo-resident artificial intelligence platform—relies on a rigid adherence to its constitutional documents and an increasingly sophisticated set of operational logic. As the system transitions from ad-hoc agent utilization to a standardized "Pointer-First" constitution, the governance mechanisms must evolve to enforce strict separation of concerns, linguistic precision, and auditability.

This report analyzes the requirements for **Dispatch Packet OPS-0042 (DP-OPS-0042)**, which seeks to certify the Agent subsystem. This certification parallels the rigor applied to the Skills subsystem in the preceding cycle (DP-OPS-0041). The primary objective is to harden the agent definition files (R-AGENT-01 through R-AGENT-06), synchronize the agent registry, and significantly strengthen the harvester algorithm responsible for identifying new agent candidates from the repository's operational exhaust.

The analysis indicates that while task and skill harvesting rely on high-frequency repetition, agent harvesting requires a "Pattern Density" heuristic suitable for low-frequency, high-complexity role emergence. This report provides a comprehensive architectural breakdown of the current system state, the theoretical underpinnings of the pointer-first constitution, and the detailed execution plan required to finalize DP-OPS-0042.

2. Constitutional Framework and Governance Logic

The Stela platform operates under a unique governance model defined by the **Policy of Truth (PoT)**. Unlike traditional software repositories where documentation is secondary to code, Stela elevates PoT.md to the status of physical law. Understanding this hierarchy is essential for analyzing the requirements of DP-OPS-0042.

2.1 The Hierarchy of Truth

The repository functions as a hierarchical system of reality definition, enforced by the "Source of Truth Read-in Order".[1] This order dictates the priority of conflicting data points and establishes the cognitive baseline for any AI agent interacting with the system.

Rank	Surface	Function	Nature

1	PoT.md	**The Constitution**. Defines physical laws, staffing, jurisdiction, and enforcement. It is the final authority in all disputes.	Immutable (requires governance event)
2	SoP.md	**The History**. The State of Play ledger. It records *what* shipped and *why*. It is a linear, append-only log of reality.	Mutable (Append-Only)
3	TASK.md	**The Contract**. The active work surface containing the Dispatch Packet. It is the only surface allowed to be "dirty" (uncommitted) during a work cycle.	Highly Mutable (Read/Write)
4	docs/MAP.md	**The Terrain**. A continuity map for navigating the repository context.	Derived/Static

The "Pointer-First" doctrine, a central theme of DP-OPS-0042, mandates that agents and tasks do not duplicate information found in PoT.md or SoP.md. Instead, they must strictly *point* to these canonical surfaces.[1] This reduces the risk of "Ghost Canon"—outdated rules lingering in duplicated text—and ensures that a single update to PoT.md instantly propagates governance changes across the entire agent swarm.

2.2 The Filing Doctrine

The operational integrity of Stela is maintained through a rigid "Filing Doctrine" defined in

PoT.md Section 1.1.[1] This doctrine assigns semantic meaning to directory structures, turning the file system into a functional component of the AI's logic.

- **ops/ (Run):** This directory contains the executable kernel of the platform. Binaries, manifests, and automation scripts live here. The integrity of ops/ is paramount because it contains the tools that verify the rest of the system.
- **docs/ (Explain):** This directory contains manuals and rationale. Crucially, the subdirectories docs/library/agents, docs/library/tasks, and docs/library/skills are designated as **JIT (Just-In-Time) Only**.
- **projects/ (Work):** Payload code lives here. It operates under the governance of the platform but does not influence platform logic.
- **storage/ (Trash):** Local artifacts, dumps, and temporary handoff files. These are never considered canon.

2.3 The Context Hazard Axiom

A critical constraint for DP-OPS-0042 is the "Context Hazard" axiom.[1] The PoT explicitly states:

> "Any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure."

This axiom exists to prevent context window saturation and logic loops. Agents, tasks, and skills are tools to be loaded *only when needed* (Just-In-Time). Including them in the global CONTEXT.md would force every AI session to process every available tool, degrading performance and increasing the likelihood of hallucination. DP-OPS-0042 must ensure that the agent harvester and linter (tools/lint/agent.sh) rigorously enforce this negative constraint.

3. Analysis of System State (Pre-DP-OPS-0042)

An analysis of the **State of Play (SoP.md)** reveals the trajectory leading to the current requirement. The system has undergone a series of hardening cycles aimed at establishing "Serviceability" and "Binary Enforcement."

3.1 Historical Trajectory

- **DP-OPS-0038 (Governance Hardening):** This packet replaced TRUTH.md with PoT.md, removing "ghost canon" and refactoring TASK.md into a pointer-first dashboard. This established the current constitutional baseline.[1]
- **DP-OPS-0039 (Task System Certification):** This cycle enforced explicit "Closeout" pointers in execution logic and mandated quantitative reporting. It proved that abstract governance rules could be enforced via binary linters (tools/lint/task.sh).[1]
- **DP-OPS-0041 (Skills System Overhaul):** This is the direct precedent for the current task. It certified the Skills subsystem (S-LEARN-01 through 06) as pointer-first. It aligned

the skills registry and enforcement tooling with binary gates.[1]

3.2 The Current Agent Subsystem

The repository currently contains six canonical agents, defined in docs/library/agents/ [1]:

1. **R-AGENT-01 (Architect):** Specializes in system design and scalability.
2. **R-AGENT-02 (Code-Reviewer):** Enforces quality and security in code changes.
3. **R-AGENT-03 (Doc-Updater):** Maintains documentation and codemaps.
4. **R-AGENT-04 (Integrator):** The lead planner for complex changes.
5. **R-AGENT-05 (Refactor-Cleaner):** Specializes in dead code removal.
6. **R-AGENT-06 (Security-Reviewer):** Focuses on vulnerability detection.

These agents are currently defined in markdown files. However, the rigor applied to their "Provenance" blocks and "Scope Boundaries" varies compared to the newly standardized Skills library. DP-OPS-0042 aims to close this gap.

3.3 The "Drift" Problem

The SoP.md entries indicate a persistent battle against "Drift"—the divergence between the defined constitution and the actual state of the repository. Drift manifests in several ways:

- **Linguistic Drift:** Use of contractions or subjective language in agent outputs.
- **Structural Drift:** Files appearing in storage/ that should be ignored, or files in docs/ that violate the filing doctrine.
- **Logic Drift:** Agents executing tasks without updating the SoP.md ledger.

DP-OPS-0042 must implement "Anti-Drift Governance" specifically for the agent subsystem. This involves updating tools/lint/agent.sh to mechanically verify that every agent file adheres to the pointer-first schema.

4. Harvester Logic: Algorithms for Role Emergence

One of the core requirements of DP-OPS-0042 is to "strengthen harvester algorithm for agent harvesting" while acknowledging that opportunities for agent harvesting are "far fewer" than for tasks or skills. This section analyzes the theoretical and mathematical basis for such an algorithm.

4.1 The Frequency/Complexity Trade-off

Tasks and skills are high-frequency, low-complexity entities. A "Verification Loop" skill (S-LEARN-01) might be used dozens of times a week. This high frequency makes simple repetition counting a valid heuristic for harvesting: if a sequence of commands is repeated N times, it is a candidate for a Skill.

Agents, however, represent *roles*—clusters of capabilities, permissions, and specialized context. They emerge not from simple repetition, but from **Pattern Density** within complex workflows. An agent is defined by a consistent *set* of tools and a specific *perspective* on the codebase.

4.2 The Pattern Density Heuristic

To algorithmically identify agent candidates, the agent.sh script must move beyond simple frequency counts. We propose a **Pattern Density (P_d)** heuristic:

$$P_d = \frac{C_t \times C_p}{F_{dp}}$$

Where:

- C_t is the cluster of distinct tools used (e.g., grep, verify.sh, lint/truth.sh).
- C_p is the cluster of distinct pointers accessed (e.g., PoT.md, security/policy.md).
- F_{dp} is the frequency of Dispatch Packets where this specific cluster appears.

Unlike skills, where high F_{dp} is the trigger, for agents, we look for high C_t and C_p values that persist across *different* objectives. If a specific set of tools (e.g., specific linters and verify scripts) is consistently paired with a specific set of governance docs (e.g., PoT.md and GOVERNANCE.md) but applied to *different* problems, this indicates a specialized **Role** (an Agent) rather than a reusable **Procedure** (a Skill).

4.3 Friction Context and Churn

The harvester must also account for **Friction Context**—areas of the codebase that generate high error rates or repeated failures.

- **Hot Zones:** Files with high modification frequency (git log churn).
- **Friction Points:** Steps in DPs that frequently fail verification (RESULTS.md analysis).

If a specific "Hot Zone" (e.g., ops/bin/llms) consistently requires intervention using a specific toolset, the harvester should propose an agent specialized for that zone (e.g., R-AGENT-07: Context-Bundler).

4.4 Implementation in agent.sh

The ops/lib/scripts/agent.sh script must be updated to:

1. **Parse SoP.md**: Extract the last 30 DPs to build a dataset of objectives and outcomes.
2. **Analyze RESULTS.md Artifacts**: Scan storage/handoff/ for command execution logs.

3. **Identify Tool Clusters**: Group commands that are frequently executed in sequence within the same session.
4. **Score Candidates**: Apply the Pattern Density logic.
    - *High Density, Low Frequency* -> **Agent Candidate**.
    - *Low Density, High Frequency* -> **Skill Candidate** (handoff to skill.sh).
5. **Generate Drafts**: Produce a markdown draft in storage/handoff/ populated with the detected toolchain and scope boundaries.

5. Architectural Certification of the Agent Subsystem

DP-OPS-0042 requires "mirroring the rigor" of the skills overhaul. This implies a specific set of certification actions for R-AGENT-01 through R-AGENT-06.

5.1 Pointer-First Refactoring

Current agent definitions may still contain "narrative bloat"—descriptions of *how* to do things rather than *pointers* to where the instructions live.

- **Old State:** "Run the verification script to check for errors."
- **Certified State:** "Execute: tools/verify.sh. Reference: docs/MANUAL.md Section 3."

The certification process must iterate through each agent file and replace narrative instructions with canonical pointers. This ensures that if tools/verify.sh changes, the agent definition does not become stale.

5.2 Provenance Enforcement

Every agent file must have a **Provenance Block** that ties its existence to a specific Dispatch Packet and system state.

- **Required Fields:** Captured, DP-ID, Branch, HEAD, Objective.
- **Logic:** An agent without provenance is "Ghost Logic." It exists without a recorded reason. DP-OPS-0042 must update the provenance of all agents to reflect their recertification under the new constitution.

5.3 Linter Hardening (tools/lint/agent.sh)

The repo-gates workflow relies on tools/lint/agent.sh to police the agent library. To mirror the rigor of the skills system, this linter must be updated to enforce:

1. **Existence of Pointers:** Verify that every file path mentioned in an agent definition actually exists (No Dead Ends).
2. **Context Hygiene:** Fail if an agent references docs/library/ recursively.
3. **Section Compliance:** Ensure strictly named sections (## Provenance, ## Role, ## Pointers, ## Scope Boundary).
4. **No Contractions:** Enforce the "Linguistic Precision" standard (e.g., reject "can't",

"don't").

6. Developing Dispatch Packet OPS-0042

Based on the analysis above, we can now construct the logical flow and content of DP-OPS-0042.

6.1 Logic and Constraints

The construction of the DP must adhere to the **Behavioral Logic Standard** [1]:

- **Absolute Literalism:** The prompt explicitly asks to "DRAFT DP-OPS-0042 based on DP-OPS-0041." This means the structure must match the TASK.md template exactly.
- **Allowlist Constraint:** The prompt includes an addendum regarding llms command invocations. Since this DP involves "context bundle refresh" (part of the system overhaul), the allowlist *must* include llms.txt, llms-small.txt, llms-full.txt, llms-ops.txt, and llms-governance.txt.
- **Reuse-First Discipline:** We must use ops/lib/scripts/agent.sh for the harvesting logic rather than inventing a new script.

6.2 Scope Boundary Analysis

The scope of this DP is broad but defined:

- **Primary Target:** The Agent Subsystem (docs/library/agents/, docs/ops/registry/AGENTS.md).
- **Tooling:** ops/lib/scripts/agent.sh (Harvester), tools/lint/agent.sh (Policing).
- **Governance:** TASK.md (Active Contract), SoP.md (History).
- **Context:** llms.txt and associated bundles (Refresh).

The **Stop Condition** must be explicit: "Any edit outside the Allowlist is a system failure. STOP immediately." This prevents "scope creep" where an agent might attempt to "fix" a project file while certifying the platform.

6.3 Execution Plan Derivation

1. **Freshness Gate:** We must validate we are on main at 5b51900d.[1]
2. **Harvester Upgrade:** This is the creative logic step. We need to define the update to agent.sh to include the "Pattern Density" logic discussed in Section 4.
3. **Linter Upgrade:** Update tools/lint/agent.sh to strictly enforce the schema.
4. **Refactor/Certify:** The bulk of the work. Edit R-AGENT-01 through 06 to be pointer-first.
5. **Verification:** Run the updated linter and the global verify script.
6. **Closeout:** Update SoP.md and generate the llms bundles.

7. Risk Assessment and Mitigation

7.1 Risk: Recursive Context Loading

There is a risk that by strengthening the agents, we inadvertently create a loop where Agent A calls Agent B, which calls Agent A.

- **Mitigation:** The "Context Hazard" check in tools/lint/agent.sh must be absolute. Agents cannot "load" other agents. They can only be "orchestrated" by a Task. The DP must enforce this distinction.

7.2 Risk: Hallucinated Capabilities

If an agent definition points to a tool that doesn't exist (e.g., ops/bin/magic-fix), the agent will hallucinate the output.

- **Mitigation:** The tools/lint/agent.sh update must include a "Dead Pointer" check that validates every file path in the ## Pointers section against the actual file system.

7.3 Risk: Semantic Collision

As the harvester becomes more aggressive, it might propose an agent that duplicates an existing one (e.g., "Architecture-Reviewer" vs "Architect").

- **Mitigation:** The agent.sh script must check the docs/ops/registry/AGENTS.md file for semantic similarity before proposing a new draft.

8. Detailed Specification for DP-OPS-0042

The following section translates the architectural analysis into the specific text required for the Dispatch Packet.

8.1 Header and Metadata

- **ID:** DP-OPS-0042
- **Branch:** work/agent-system-certification-0042 (Proposed)
- **Objective:** Certify Agent subsystem, harden harvester, and enforce pointer-first constitution.

8.2 The Allowlist

This is critical for the "Anti-Drift Governance." We must list every file the Contractor is authorized to touch.

- All R-AGENT-*.md files.
- ops/lib/scripts/agent.sh.
- tools/lint/agent.sh.
- docs/library/AGENTS.md (Ledger).
- docs/ops/registry/AGENTS.md (Registry).
- SoP.md, TASK.md (Governance).

- llms.txt and bundles (Root outputs).

8.3 The Execution Logic

The steps must be deterministic.

1. **Verification of State:** Ensure the repo is clean.
2. **Tooling Upgrade:** Modify the scripts first. This ensures that when we refactor the agents, we have the tools to verify them immediately.
3. **Refactoring:** Process the agents one by one.
4. **System Check:** Run the full lint suite.
5. **Context Refresh:** Regenerate the LLM bundles to reflect the new reality.

9. Conclusion

The transition to a pointer-first agent constitution represents a maturation of the Stela platform. By replacing narrative instructions with rigid pointers to canonical truth, we reduce the surface area for hallucination and drift. The rigor applied to DP-OPS-0042 ensures that the "Staffing" of the AI system—its agents—are as rigorously defined and maintained as the code they operate on.

The harvester logic upgrade addresses the specific challenge of "Role Emergence" in a stateless system. By moving from frequency-based to density-based heuristics, we enable the system to detect nuanced architectural needs that would otherwise be missed by simple repetition counting.

The following Dispatch Packet is the executable embodiment of this analysis.

DP-OPS-0042: Agent System Certification and Harvester Logic Hardening

DP-OPS-0042: Agent System Certification and Harvester Logic Hardening

1. Administrative Data

- **Status:** DRAFT
- **Owner:** Integrator
- **Date:** 2026-02-10
- **Base Branch:** main
- **Work Branch:** work/agent-system-certification-0042
- **Base HEAD:** 5b51900d

2. Logic Pointers (The Law)

- **Constitution:** PoT.md (Primary Authority).
- **History:** SoP.md (Ledger).
- **Active Contract:** TASK.md (This Document).
- **Execution Standard:** docs/MANUAL.md.

3. Dispatch Packet (DP)

3.1 Scope Boundary

Objective: Certify the Agent subsystem as pointer-first and serviceable by aligning R-AGENT-01 through R-AGENT-06, the agent registry, and enforcement tooling with binary gates. Strengthen the ops/lib/scripts/agent.sh harvester algorithm to detect emergence patterns and establish a mandatory harvesting schedule.

Allowlist (Hard Gate):

- ops/lib/scripts/agent.sh
- tools/lint/agent.sh
- docs/library/AGENTS.md
- docs/ops/registry/AGENTS.md
- docs/library/agents/R-AGENT-01.md
- docs/library/agents/R-AGENT-02.md
- docs/library/agents/R-AGENT-03.md
- docs/library/agents/R-AGENT-04.md
- docs/library/agents/R-AGENT-05.md
- docs/library/agents/R-AGENT-06.md
- ops/bin/llms
- llms.txt
- llms-small.txt
- llms-full.txt
- llms-ops.txt
- llms-governance.txt
- TASK.md

- SoP.md

Stop Condition: Any edit outside the Allowlist is a system failure. STOP immediately.

3.2 Execution Plan

1. **Context Load:** Verify OPEN artifact matches git branch main and HEAD 5b51900d.
2. **Harvester Logic Hardening:**
   - Update ops/lib/scripts/agent.sh to implement a "Pattern Density" heuristic.
   - **Requirement:** If a specific combination of tools and pointers appears in 3 or more DPs, flag the combination for agent candidacy.
   - **Scheduling:** Update docs/library/AGENTS.md to mandate an "Agent Harvest Check" at the close of any DP classified as "System Overhaul" or "Architecture Refactor".
3. **Linter Hardening:**
   - Update tools/lint/agent.sh to enforce strict existence of ## Provenance, ## Pointers, and ## Scope Boundary sections in all R-AGENT-*.md files.
   - **Metric:** Ensure 0 failures and 0 warnings allowed for agent definition files.
4. **Agent Certification:**
   - Iterate through R-AGENT-01 to R-AGENT-06.
   - Update the **Provenance** block in each file to reflect "Recertification via DP-OPS-0042".
   - Verify that every toolchain pointer listed in Authorized toolchain resolves to an existing file on disk.
5. **Registry Synchronization:**
   - Verify docs/ops/registry/AGENTS.md contains exact matches for all agent IDs and filenames.
6. **Context Refresh:**
   - Run ops/bin/llms to regenerate llms.txt and scope bundles.
   - **Constraint:** This command affects repository root files; verify allowlist compliance before execution.
7. **Verification:**
   - Run bash tools/lint/agent.sh.
   - Run bash tools/lint/library.sh.
   - Run bash tools/verify.sh.
   - Capture outputs in RESULTS.

3.3 Skill Integration

Harvest and promote via ops/lib/scripts/skill.sh.

Constraint: Do not edit docs/library/SKILLS.md manually.

4. Closeout (Mandatory)

Requirement: Populate storage/handoff/DP-OPS-0042-RESULTS.md and append the following block to TASK.md Section 5.

Mandatory Closing Block

Constraint: Final Squash Stub must differ significantly from Primary Commit Header.

1. **Primary Commit Header:** DP-OPS-0042 agent system certification and harvester hardening
2. **Pull Request Title:** DP-OPS-0042 Agent System Overhaul
3. **Pull Request Description:**

   Summary
   - Hardened ops/lib/scripts/agent.sh with Pattern Density heuristics for low-frequency agent harvesting.
   - Updated tools/lint/agent.sh to strictly enforce provenance and scope boundaries.
   - Recertified R-AGENT-01 through R-AGENT-06 with updated provenance blocks.
   - Refreshed context bundles via ops/bin/llms.

   Testing
   - **bash tools/lint/agent.sh (PASS)**
   - bash tools/lint/library.sh (PASS)
   - bash tools/verify.sh (PASS)
   - ops/bin/llms (Generated 5 bundles)
4. **Final Squash Stub:** Enforce binary gates for agent subsystem and harden harvest logic
5. **Extended Technical Manifest:**
   - ops/lib/scripts/agent.sh
   - tools/lint/agent.sh
   - docs/library/AGENTS.md
   - docs/ops/registry/AGENTS.md
   - docs/library/agents/R-AGENT-*.md
   - llms.txt
   - TASK.md
   - SoP.md
6. **Review Conversation Starter:**
   Does the Pattern Density heuristic in the agent harvester correctly balance the low frequency of agent emergence with the need for rigorous capture?

Works cited

1. dump-full-main-5b51900d.txt